

Hoja de trabajo 5 – Orquestación

Diego Andre Calderón Salazar
241263

Hoja de trabajo #5 – Orquestación de Sistemas Multiagente (MAS)

1. Resumen y Configuración del Sistema

El proyecto implementa un sistema conversacional para la empresa Parachute S.A. que atiende dudas institucionales, consulta pronósticos meteorológicos en tiempo real y gestiona la reserva de citas para saltos en paracaídas.

- **Modelo de lenguaje:** `openai/gpt-oss-20b` alojado en NVIDIA NIM, consumido mediante OpenAI Agents SDK y AsyncOpenAI.
- **Base de datos:** PostgreSQL 16 con extensión `pgvector` para la recuperación de 120 preguntas frecuentes mediante índice HNSW, y una tabla transaccional para citas con restricción de unicidad por franja horaria.
- **Servicios externos:** API de Open-Meteo con agregación diurna y evaluador de seguridad determinista en Python.

2. Pregunta 1: ¿Qué arquitectura resuelve mejor este problema? ¿Por qué?

Comparativa y Resultados Observados

Se evaluaron tres arquitecturas bajo las mismas condiciones y consultas de prueba. Los tiempos de respuesta y llamadas al modelo fueron los siguientes:

Arquitectura	Latencia FAQ	Latencia Flujo Combinado	Llamadas LLM por Turno	Responsable de Respuesta
Centralizada	112.45 s	185.30 s	3 - 4	Orquestador central

Arquitectura	Latencia FAQ	Latencia Flujo Combinado	Llamadas LLM por Turno	Responsable de Respuesta
Descentralizada	204.71 s	290.15 s	2 - 6	Especialista según handoff
Jerárquica	307.62 s	389.40 s	5 - 8	Orquestador raíz

Análisis Técnico

1. Arquitectura Centralizada (Solución más adecuada para el alcance actual):

Presenta el mejor desempeño y simplicidad operativa. Al contar con un único punto de entrada, el orquestador descompone la solicitud del usuario, consulta a los especialistas mediante herramientas (`as_tool`) y reúne los resultados en una respuesta clara. Este esquema minimiza las rondas de inferencia frente a estructuras con niveles intermedios, facilitando además la trazabilidad de las decisiones.

2. Arquitectura Jerárquica (Alternativa viable para escenarios de mayor escala):

Organiza las tareas en dos niveles de supervisión: Conocimiento (preguntas frecuentes y políticas) y Operaciones (clima, seguridad y reservas). El supervisor de operaciones hace cumplir de manera estructural la secuencia obligatoria de consultar el clima y verificar la viabilidad antes de agendar. Sin embargo, en el entorno actual introduce una penalización considerable en tiempo de respuesta debido a las llamadas anidadas entre niveles. Su aplicación tiene sentido si el sistema incorpora nuevos departamentos o una cantidad significativamente mayor de herramientas.

3. Arquitectura Descentralizada:

Resulta la menos conveniente para este caso. La necesidad de validar condiciones meteorológicas antes de reservar exige transferencias de control estrictas entre pares. Cuando el usuario cambia de tema o realiza preguntas compuestas, la falta de un coordinador central provoca transferencias reiteradas y

pérdida de contexto, elevando tanto la latencia como el riesgo de inconsistencias.

3. Pregunta 2: ¿Considera que es necesario utilizar un sistema multiagente en este caso? ¿Por qué?

Para el alcance actual del proyecto, **un sistema multiagente no es necesario** y resulta ineficiente en términos de tiempo de respuesta y consumo de recursos.

Fundamentos Técnicos

1. Naturaleza determinista de las operaciones críticas:

La verificación de seguridad (evaluación de velocidad del viento, ráfagas, precipitación y cobertura nubosa) no requiere inferencia por parte de un modelo de lenguaje. Se resuelve de forma exacta mediante código determinista en Python (`core/safety.py`). De igual manera, el control de disponibilidad de horarios y la prevención de sobrecupos se resuelven mediante transacciones y restricciones de unicidad en la base de datos relacional.

2. Eficiencia de un agente único con herramientas:

Un solo agente provisto de cuatro funciones bien definidas (`buscar_faqs`, `consultar_clima`, `consultar_disponibilidad` y `agendar_cita`) resolvería los mismos flujos en una o dos llamadas por turno. Esto reduciría la latencia de minutos a pocos segundos, con un menor gasto de tokens y menor probabilidad de fallos en llamadas remotas.

Condiciones para Adoptar un Sistema Multiagente

El uso de arquitecturas multiagente se justifica únicamente cuando se presentan necesidades organizacionales o técnicas específicas:

- **Aislamiento de permisos:** Cuando ciertos agentes deban manejar credenciales o datos sensibles (por ejemplo, registros de clientes o pagos) a los que otros agentes no deben tener acceso.
- **Sobrecarga de contexto:** Cuando el catálogo de herramientas y directivas sea tan extenso que degrade la capacidad del modelo

para seleccionar la herramienta correcta en un solo prompt.

- **Desarrollo desacoplado:** Cuando equipos distintos mantengan dominios funcionales independientes sin intervenir en las instrucciones de otros módulos.
-

4. Decisiones Técnicas y Control de Restricciones

Tratamiento de la Ventana Meteorológica

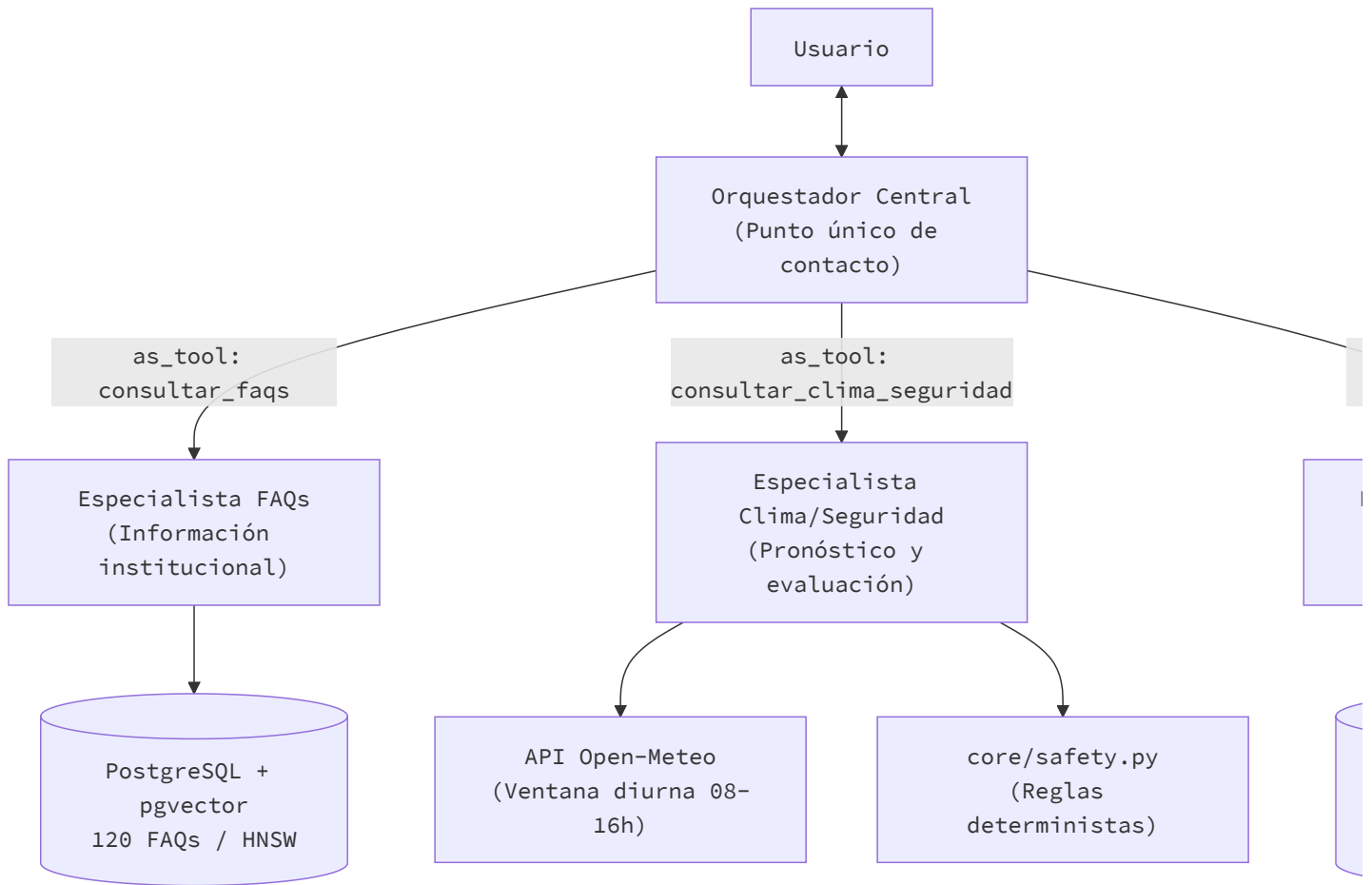
La zona de salto se ubica en la costa sur de Guatemala (`14.013722, -90.771611`). Durante los meses de septiembre y octubre, la región presenta precipitaciones constantes en horas de la tarde y noche. Si se evaluara el pronóstico diario acumulado (24 horas), la totalidad de las fechas se clasificarían como no aptas. Por esta razón, el sistema analiza únicamente la ventana diurna de actividad (08:00 a 16:00) mediante los registros horarios de Open-Meteo, garantizando evaluaciones coherentes con los horarios reales de salto. Para pruebas automatizadas, se implementó además un mecanismo de simulación controlada (`--mock-clima`).

Manejo de Inconsistencias en el Corpus

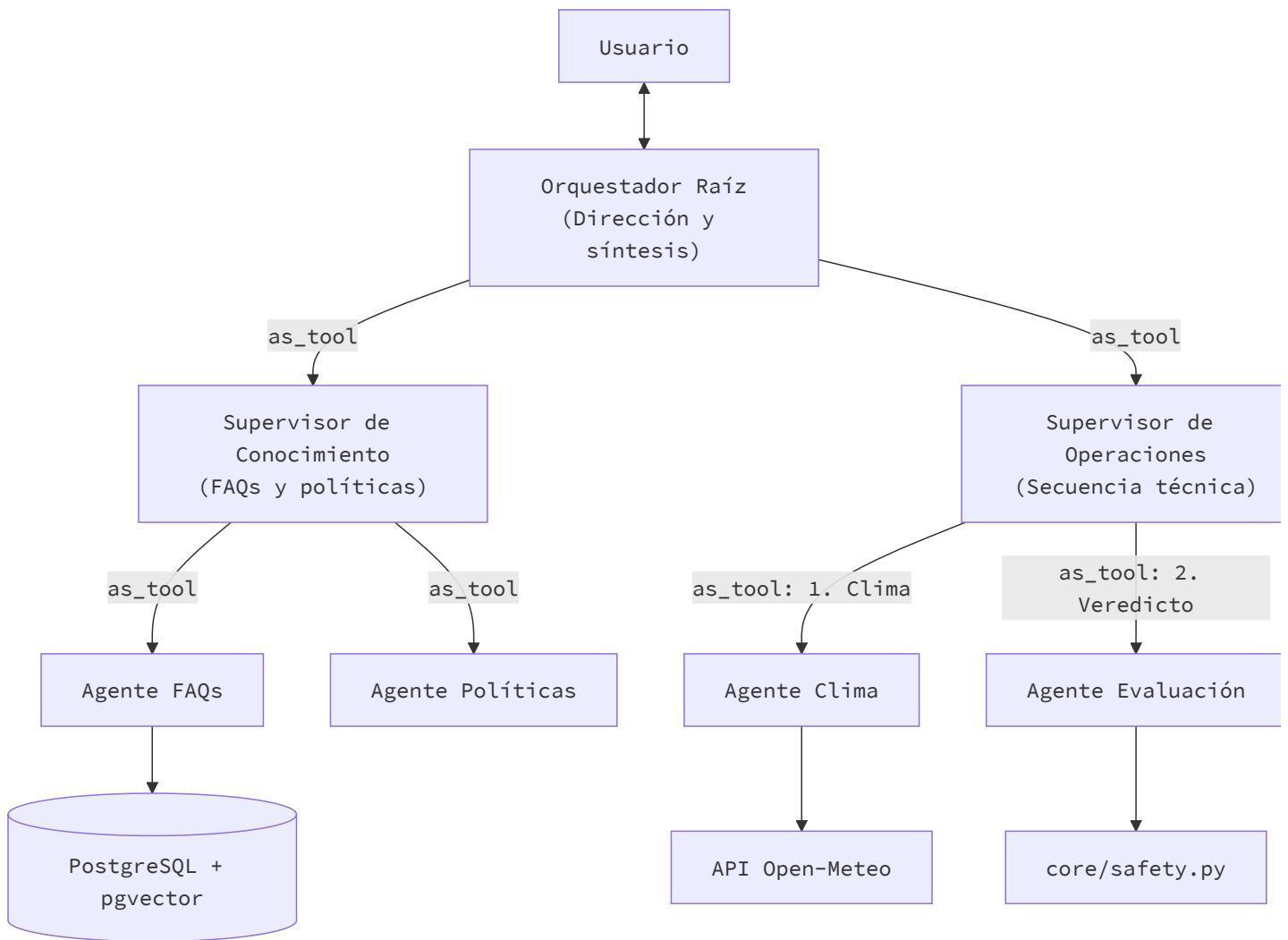
El archivo de preguntas frecuentes contiene entradas con información contradictoria o ambigua respecto a límites operativos (por ejemplo, respuestas que confunden la velocidad terminal en caída libre con el límite de viento para operar). Para evitar que el modelo interprete estos textos como normas de seguridad, dichas entradas fueron catalogadas como inconsistentes en la capa de datos (`core/faq_store.py`). El sistema exige que cualquier criterio de autorización provenga de la función determinista de evaluación y no del contenido textual de las preguntas frecuentes.

5. Diagramas de Arquitectura

Arquitectura Centralizada



Arquitectura Jerárquica



Arquitectura Descentralizada

